

CORPORACIÓN UNIVERSITARIA REMINGTON

UNIREMINGTON

ARQUITECTURA ORIENTADA A MICROSERVICIOS

Ejercicio práctico de comunicación entre servicios mediante API REST

Estudiante

Omar Antonio Mosquera Palacios

Asignatura

Lenguaje de Programación Avanzado 2

Tecnologías

Java 17 · Spring Boot 3.2.5

20 de septiembre de 2026

Documento final de entrega

1. Comparación: monolito vs. microservicios

La arquitectura monolítica concentra las funcionalidades principales de una aplicación dentro de una misma unidad de despliegue. En una arquitectura de microservicios, la solución se divide en servicios independientes que se comunican mediante interfaces, por ejemplo APIs REST.

Criterio	Arquitectura monolítica	Arquitectura de microservicios
Estructura	Una aplicación contiene las principales funcionalidades.	La aplicación se divide en servicios independientes.
Despliegue	Normalmente se despliega la aplicación como una unidad.	Cada servicio puede desplegarse de forma independiente.
Escalabilidad	Puede requerir escalar toda la aplicación.	Se puede escalar un servicio específico según la necesidad.
Mantenimiento	Un cambio puede afectar una parte importante del sistema.	Los cambios pueden concentrarse en el servicio correspondiente.
Comunicación	Las funciones se comunican dentro de la misma aplicación.	Los servicios se comunican mediante APIs o mecanismos de mensajería.
Complejidad	La operación inicial suele ser más sencilla.	Aumenta la complejidad por la coordinación y monitoreo de varios servicios.

2. Breve ensayo: adopción de microservicios

Las empresas de tecnología que manejan productos digitales de gran tamaño pueden adoptar arquitecturas de microservicios para organizar sistemas complejos en componentes más pequeños. En lugar de mantener todas las funcionalidades dentro de una sola aplicación, cada servicio puede concentrarse en una responsabilidad determinada y comunicarse con otros servicios mediante interfaces definidas.

Una de las razones para utilizar este enfoque es la posibilidad de trabajar con diferentes partes del sistema de manera más independiente. Cuando una funcionalidad necesita cambios o una capacidad de procesamiento diferente, el equipo puede intervenir principalmente en el servicio relacionado. Esto también permite organizar el desarrollo por responsabilidades y facilita que diferentes equipos trabajen sobre componentes concretos.

Sin embargo, utilizar microservicios no significa que una aplicación automáticamente sea más sencilla. La solución debe manejar la comunicación entre servicios, los errores de red, el monitoreo, la configuración y la seguridad. También es necesario definir claramente las responsabilidades de cada servicio para evitar una arquitectura difícil de mantener. Por esta razón, la adopción de microservicios debe responder a las características y necesidades reales del proyecto.

3. Desarrollo práctico

Para la actividad se construyeron dos servicios independientes utilizando Java 17 y Spring Boot 3.2.5. El primer servicio administra usuarios y se ejecuta en el puerto 8081. El segundo administra pedidos y se ejecuta en el puerto 8082.

3.1. Servicios implementados

usuarios-service: proporciona endpoints para consultar usuarios.

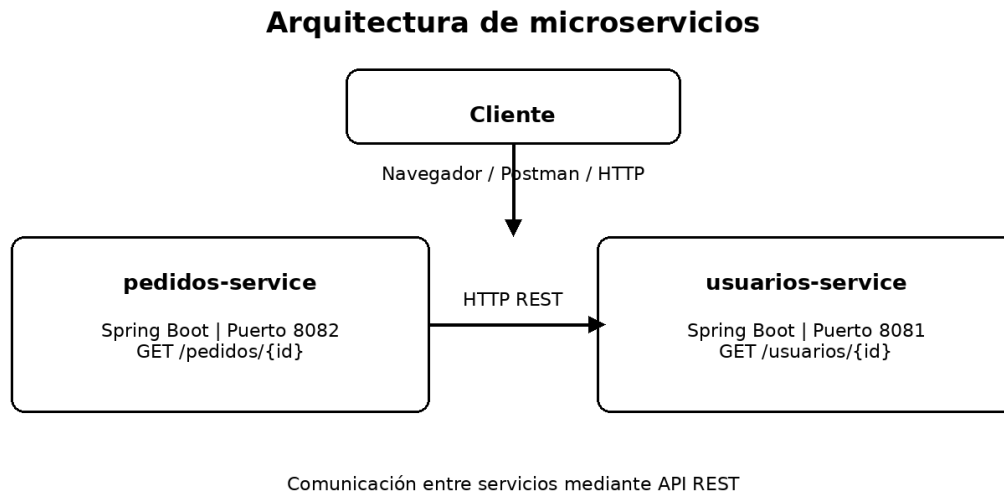
pedidos-service: proporciona endpoints para consultar pedidos y solicita información de usuarios mediante HTTP REST.

3.2. Endpoints utilizados

Método	Endpoint	Servicio	Función
GET	/usuarios	usuarios-service	Consulta todos los usuarios.
GET	/usuarios/1	usuarios-service	Consulta un usuario por ID.
GET	/pedidos	pedidos-service	Consulta todos los pedidos.
GET	/pedidos/1	pedidos-service	Consulta un pedido y obtiene la información del usuario asociado.

3.3. Diagrama de arquitectura

El cliente realiza solicitudes al servicio de pedidos. Cuando pedidos necesita los datos de un usuario, realiza una solicitud REST al servicio de usuarios.



4. Evidencia de funcionamiento

Los dos servicios fueron ejecutados localmente y se realizaron consultas a sus endpoints. La prueba principal consistió en consultar el pedido 1 desde pedidos-service y observar que la respuesta incluyera los datos del usuario obtenidos desde usuarios-service.

Prueba principal

Invoke-RestMethod http://localhost:8082/pedidos/1

(Invoke-RestMethod http://localhost:8082/pedidos/1) | ConvertTo-Json -Depth 5

Resultado observado

```
{
  "id": 1,
  "producto": "Laptop",
  "cantidad": 1,
  "usuario": {
    "id": 1,
    "nombre": "Omar",
    "correo": "omar@example.com"
  }
}
```

La respuesta muestra el pedido y, dentro del mismo resultado, los datos del usuario. Esto permite demostrar la comunicación entre los dos servicios mediante una API REST.

Organización del código

El proyecto contiene los directorios usuarios-service y pedidos-service. Cada servicio posee su propio pom.xml, clases de aplicación, controladores, modelos y configuración. En pedidos-service se utiliza la clase UsuarioClient para realizar la consulta al servicio de usuarios.

5. Entregables y enlaces

Código fuente en GitHub

<https://github.com/Marelissofiaescobar/proyecto-microservicios-spring-boot>

Video de sustentación en YouTube

<https://youtu.be/YL2WvooAoss>

El video presenta la estructura del proyecto, la ejecución de los dos servicios, las pruebas de los endpoints, la comunicación REST y una explicación general de la arquitectura.

6. Conclusiones

La implementación permitió demostrar de manera práctica una arquitectura básica de microservicios con Spring Boot. Los servicios de usuarios y pedidos funcionan de forma independiente y utilizan diferentes puertos.

La comunicación entre servicios se realizó mediante HTTP REST. La consulta del pedido permitió comprobar que pedidos-service puede solicitar información al servicio de usuarios y utilizarla para construir la respuesta.

El ejercicio también permitió identificar que los microservicios aportan independencia entre componentes, pero requieren mayor atención a la comunicación, configuración, monitoreo y manejo de errores.

7. Referencias

Spring. (s. f.). Spring Boot documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring. (s. f.). Building a RESTful Web Service. <https://spring.io/guides/gs/rest-service/>

Microsoft. (s. f.). Visual Studio Code documentation. <https://code.visualstudio.com/docs>